Università
degli Studi
di Padova

Dipartimento
di Fisica
e Astronomia
Galileo Galilei

Charm hadrons reconstruction in ALICE

Pieripolli Leonardo, Pirazzo Tommaso,
Ponchio Mattia, Secco Benedetto

10th February 2026

Objective

Reconstruct the decay of the two charm hadrons:

$$D^0 \to K^-\pi^+ \quad \text{BR: } (3.947 \pm 0.030)\% \text{ [1]}$$

$$\Lambda_c^+ \to pK^-\pi^+ \quad \text{BR: } (6.28 \pm 0.32)\% \text{ [2]}$$

[1] https://pdg.lbl.gov/2022/listings/rpp2022-list-D-zero.pdf
[2] https://pdg.lbl.gov/2019/listings/rpp2019-list-lambdac-plus.pdf

O2collision

Data on the collisions, specifically about the Primary Vertex.

1. **fPosX, fPosY, fPosZ**: Coordinates of the PV in the *global reference system*.
2. **Row index**: Variable referenced by the other tables needed for the matching.

Data Structure

O2collision

Data on the collisions, specifically about the Primary Vertex.

1. **fPosX, fPosY, fPosZ**: Coordinates of the PV in the *global reference system*.
2. **Row index**: Variable referenced by the other tables needed for the matching.
3. (MC only) **fPosX, fPosY, fPosZ**: Global coordinates of the collision point in O2mccollision.

Data Structure

O2genparticles

Data on filtered generated particles in the Monte Carlo simulation.

1. **fPdgCode**: PDG code of the generated particle:
   $D^0 \rightarrow 421$ $\overline{D^0} \rightarrow -421$ $\Lambda^{+(-)} \rightarrow (-)4211$
2. **fMainMotherPt, fMainMotherY**: Transverse momentum p_T and rapidity Y for the D^0 (or $\Lambda^{\pm}$) particle.
3. **fMainBeautyAncestorPdgCode**: PDG code of beauty particles for cases in which the charm hadrons derive from beauty decay, 0 otherwise.

Data Structure

O2filtertrack

Data on all the tracks generated from the respective collisions.

1. **fIndexCollisions**: References the row of *O2collision* corresponding to the generating collision for this track.
2. **fAlpha**: Angle between the global reference system and the tracking reference system.
3. **fX, fY, fZ**: Coordinates of the Point of Closest Approach to the PV in the *tracking system*.

In order to convert from track coordinates to global coordinates:

$$\begin{pmatrix} x_{global} \\ y_{global} \end{pmatrix} = \begin{bmatrix} \cos\alpha & -\sin\alpha \\ \sin\alpha & \cos\alpha \end{bmatrix} \begin{pmatrix} x_{track} \\ y_{track} \end{pmatrix}$$

Data Structure

O2filtertrackextr

Additional data on the tracks: each row references the corresponding row of *O2filtertrack* with the same index.

1. **fPt**: Component of the momentum in the transverse plane (XY).Inversely proportional to the curvature for charged particles.
2. **fEta**: Pseudorapidity $\eta = \frac{1}{2} \ln \frac{p+p_z}{p-p_z} = -\ln \tan \theta$ (θ is the polar angle).
3. **fCharge**: Electrical charge of the particles in units of e.
4. **fDcaXY**: Track impact parameter (w.r.t. the PV) in the XY plane.
5. **fNsigmaTPC[pi, ka, pr]**: $n\sigma = \frac{\text{measured - expected}}{\sigma}$ for the indicated particle for the Time Projection Chamber.
6. **fNsigmaTOF[pi, ka, pr]**: Same as the previous but for the Time Of Flight.

Data Structure

O2filtertrackmc

Contains simulation information of the particle associated to the reconstructed track.

1. **fPdgCode:**: PDG code of the track particle: $\pi^{+(-)}: (-)211$; $K^{+(-)}: (-)321$; $p^{+(-)}: (-)2212$
2. **fMainHfMotherPdgCode**: PDG code of the mother when relevant, 0 otherwise.
3. **fMainBeautyAncestorPdgCode**: PDG code of beauty particles for cases in which the charm hadrons derive from beauty decay, 0 otherwise.
4. **fMainMotherNfinalStateDaught**: Number of final-state daughters.
5. **fMainMotherOrigIndex**: Index of the mother particle in the original simulation tree.

Example

A decay is recognized as the following only if it satisfies all conditions.

$$D^0 \to K^- \pi^+$$

1. **fPdgCode**: The couple {-321, +211}.
2. **fMainHfMotherPdgCode** is 421 for both.
3. **fMainMotherOrigIndex** is the same for both tracks.
4. **fMainMotherNfinalStateDaught** is 2 (for both).
5. **fMainBeautyAncestor** is 0 (for both).

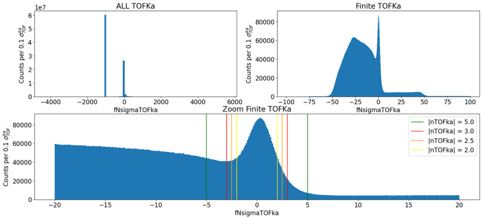
Example

A decay is recognized as the following only if it satisfies all conditions.

$$D^0 \to K^-\pi^+$$

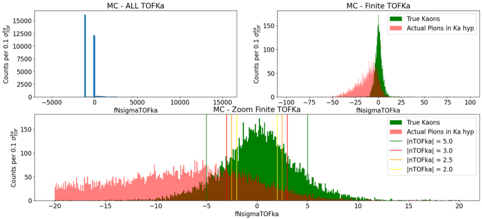
1. **fPdgCode**: The couple $\{$-321, $+$211$\}$.
2. **fMainHfMotherPdgCode** is 421 for both.
3. **fMainMotherOrigIndex** is the same for both tracks.
4. **fMainMotherNfinalStateDaught** is 2 (for both).
5. **fMainBeautyAncestor** is 0 (for both).
6. $|$fPosZ$| < 10$ (value in cm).
7. $|\eta| < 0.8$

fNsigmaTOFka




```
Unavailable TOFka: 67.60%
TOFka distribution in % (These are referred to the portion of data whose TOFka is not -999)
```

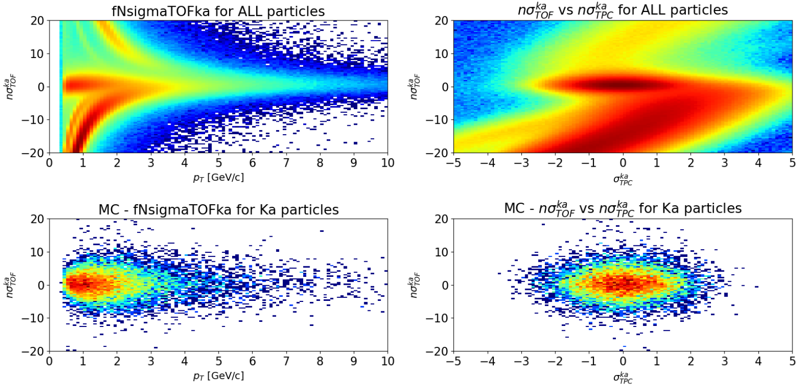
	[-0.5, 0.5]	[-1, 1]	[-1.5, 1.5]	[-2.0, 2.0]	[-2.5, 2.5]	[-3.0, 3.0]	[-4.0, 4.0]	[-5, 5]	[-7, 7]	[-10, 10]
% of Finite TOFKa	2.89	5.49	7.66	9.41	10.82	11.99	13.96	15.76	19.36	25.04
% of Total	0.94	1.78	2.48	3.05	3.5	3.88	4.52	5.11	6.27	8.11



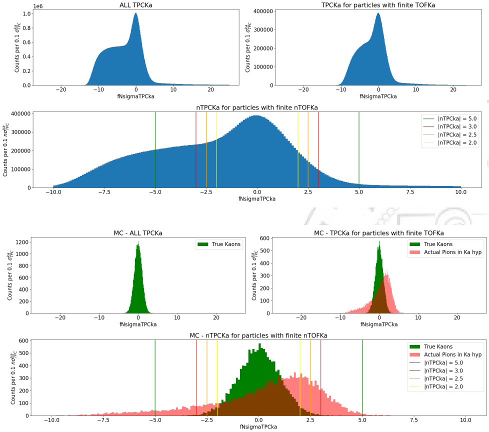

```
Unavailable TOFka: 54.62%
TOFka distribution in % (These are referred to the portion of data whose TOFka is not -999)
```

	[-0.5, 0.5]	[-1, 1]	[-1.5, 1.5]	[-2.0, 2.0]	[-2.5, 2.5]	[-3.0, 3.0]	[-4.0, 4.0]	[-5, 5]	[-7, 7]	[-10, 10]
% of Finite TOFKa in MC	11.11	21.7	32.0	41.0	48.88	55.77	66.53	74.72	83.6	88.51
% of Total	5.04	9.85	14.52	18.6	22.18	25.31	30.19	33.91	37.94	40.17

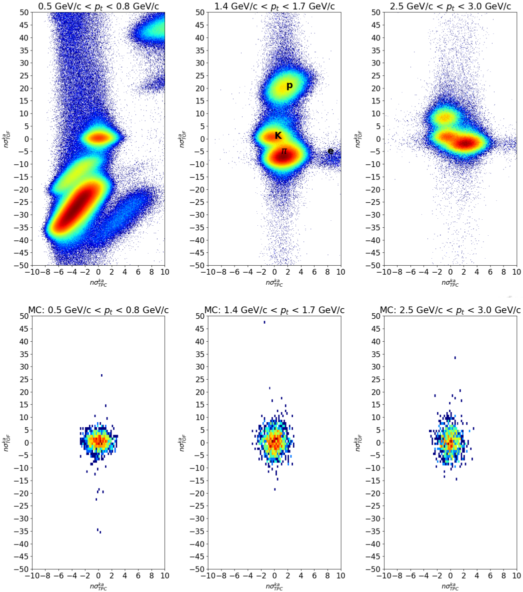
fNsigmaTOFka



fNsigmaTPCka

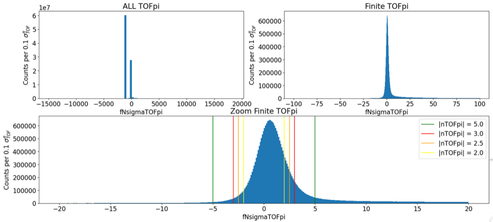


fNsigmaTOFka vs FNsigmaTPCka



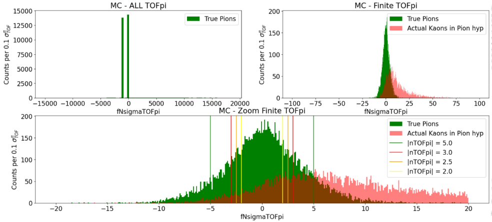
- Different particles well separated at low p_T
- Distinguishing between them based on TOF and TPC becomes more difficult as p_T increases.

fNsigmaTOFpi




```
Unavailable TOFpi: 67.60%
TOFpi distribution in % (These are referred to the portion of data whose TOFpi is not -999)
```

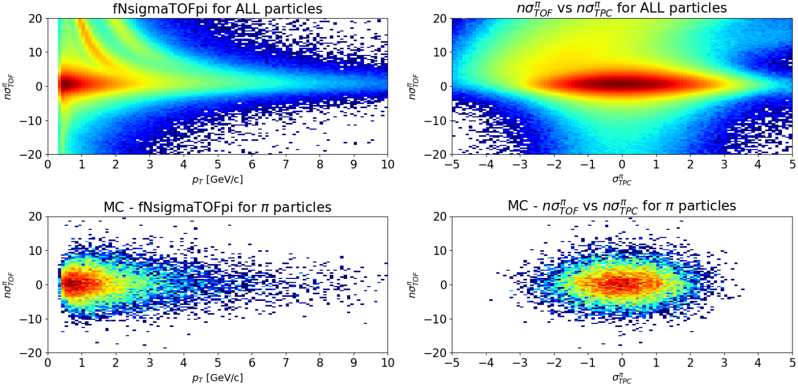
	[-0.5, 0.5]	[-1, 1]	[-1.5, 1.5]	[-2.0, 2.0]	[-2.5, 2.5]	[-3.0, 3.0]	[-4.0, 4.0]	[-5, 5]	[-7, 7]	[-10, 10]
% of Finite TOFpi	19.35	36.4	49.85	59.58	66.22	70.63	75.52	77.96	80.51	82.7
% of Total	6.27	11.79	16.15	19.3	21.46	22.89	24.47	25.26	26.09	26.8



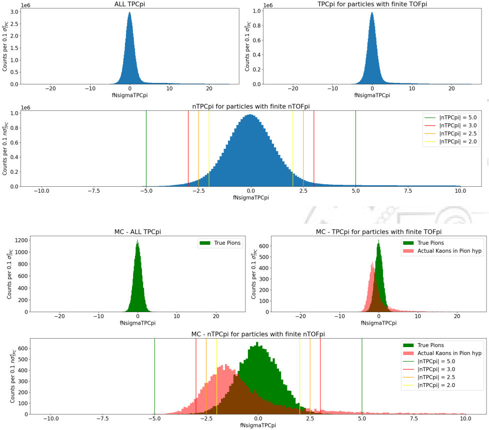

```
Unavailable TOFpi: 46.55%
TOFpi distribution in % (These are referred to the portion of data whose TOFpi is not -999)
```

	[-0.5, 0.5]	[-1, 1]	[-1.5, 1.5]	[-2.0, 2.0]	[-2.5, 2.5]	[-3.0, 3.0]	[-4.0, 4.0]	[-5, 5]	[-7, 7]	[-10, 10]
% of Finite TOFpi in MC	10.8	21.23	30.95	39.84	47.44	54.27	65.35	73.72	83.78	89.06
% of Total	5.77	11.35	16.54	21.3	25.36	29.01	34.94	39.41	44.78	47.61

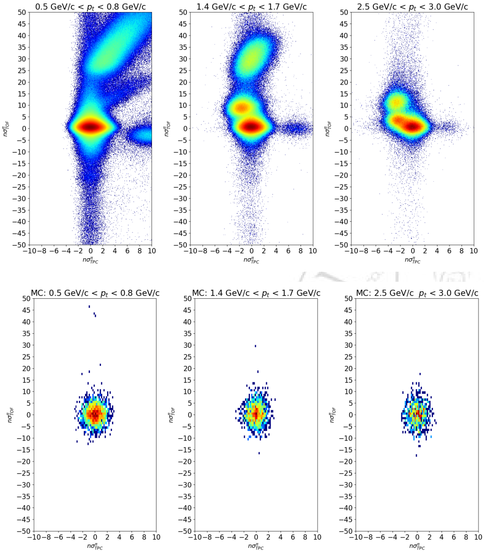
fNsigmaTOFpi



fNsigmaTPCpi



fNsigmaTOFpi vs FNsigmaTPCpi



Data Size

- $\approx$ 39GB of TTree Files in 17 chunks.
- $\approx$ 113 Millions of Collision.
- $\approx$ 603 Millions of Tracks.

Data Size

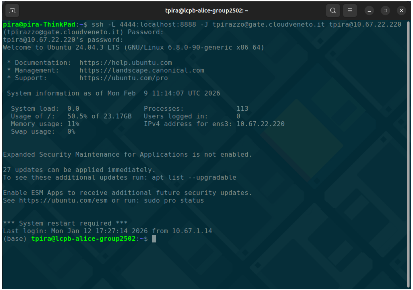
- $\approx$ 39GB of TTree Files in 17 chunks.
- $\approx$ 113 Millions of Collision.
- $\approx$ 603 Millions of Tracks.

Cannot process everything at once!

$\Downarrow$

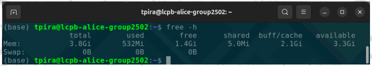
Even a single chunk of $\approx$ 2GB can be troublesome once we take into account the combinatorial for the individuation of the couples of daughter particles of D^0.

Cloud Veneto



- cpu resources

- RAM memory resources



- In order to mantain the process active even if we interrupt the ssh connection we adopted tmux.

Our approach

Since the 17 chunks are each made of hundreds of independent TDirectories, we operated the analysis on a subset and saved the results to file. This process is repeated until all data of the considered chunk is exhausted.

If there is enough RAM available, it is possible to parallelize the procedure in order to obtain a considerable speed-up (we implemented the Pool method from multiprocessing).

Code details: Parallelization

```
import multiprocessing as mp

# Define the number of processes
N_PROCESSES = 3
# Create a pool of processes.
pool = mp.Pool(N_PROCESSES)
# This can be equal to the number of processes
N_SPLITS = 9

# Number of subsets of the data to create: this is due to memory problems when doing the combinatorial.
subsets = np.array_split(range(0,N_FILES),N_SPLITS)

results = pool.starmap(compute_pairs, [(path, subsets[i]) for i in range(len(subsets))])
```

Code details: Merging

```
# Load the file.
        with uproot.open(file_path) as file:
            # Load TTrees from directories
            names_track_extr = file.keys(filter_name=r"*O2filtertrackextr")[ID_split[0]: ID_split[-1]+1]
            names_track      = file.keys(filter_name=r"*O2filtertrack")[ID_split[0]: ID_split[-1]+1]
            names_coll       = file.keys(filter_name=r"*O2collision_001")[ID_split[0]: ID_split[-1]+1]

        # Variable to correct the index of the collision (because it starts from 0 at each new TTree).
        collision_offset = 0
        # Container for the dataframes created with the pairs.
        list_of_df = []

        for i in range(len(names_coll)):

            # Read collision tree.
            df_coll = file[ names_coll[i] ].arrays(["fPosX", "fPosY", "fPosZ"], library="pd")

            # Read track and trackextr using boolean mask for hypothesis on particle identity.
            df_trackextr = file[ names_track_extr[i] ].arrays(["fPt", "fEta", "fCharge", "fDcaXY", "fNsigmaTPCpi", "fNsigmaTPCka",
                                                                "fNsigmaTOFpi", "fNsigmaTOFka"], library="pd" )
            df_track = file[ names_track[i] ].arrays(["fIndexCollisions", "fAlpha", "fX", "fY", "fZ"], library="pd")

            # Merge all rows: fIndexCollisions in O2filtertrack must match the index in O2collision_001;
            # O2filtertrack and O2filtertrackextr, instead, match 1:1 row-wise.
            df_track = pd.merge(left=df_track, right=df_coll, how="inner", left_on = "fIndexCollisions", right_index=True)
            df_trackextr = pd.merge(left=df_trackextr, right=df_track, how='inner', left_index=True, right_index=True)

            # Assign particle nature hypothesis based on the chosen cuts.
            mask_both = df_trackextr.eval(f"({Ka_hyp} & {Pi_hyp})")
            mask_Ka = df_trackextr.eval(Ka_hyp)
            mask_Pi = df_trackextr.eval(Pi_hyp)

            # From the documentation of numpy select: numpy.select(condlist, choicelist, default=0)
            # When multiple conditions are satisfied, the first one encountered in condlist is used.
            df_trackextr["Hyp"] = np.select([mask_both, mask_Ka, mask_Pi], ["Both", "Kaon", "Pion"], default="Bkg")

            # Apply the SAME filter to df_trackextr and df_track to keep them aligned (and keep only useful columns).
            mask = mask_Ka | mask_Pi
            df_trackextr = df_trackextr.loc[mask, ["fIndexCollisions","fAlpha", "fX", "fY", "fZ","fPt", "fEta", "fCharge", "fDcaXY","Hyp",
                                                   "fPosX", "fPosY", "fPosZ", "fNsigmaTPCpi", "fNsigmaTPCka", "fNsigmaTOFpi", "fNsigmaTOFka"

            # The rows where the fIndexCollision is negative have been excluded thanks to the merge on the index of collision, which can onl
            # non-negative. So, we keep only those with |fPosZ| < 10 and |pseudorapidity| < 0.8
            valid = ( df_trackextr["fPosZ"].abs() < 10 ) & ( df_trackextr["fEta"].abs() < 0.8 )
            df_trackextr = df_trackextr[valid].reset_index(drop=True)

            # Fix local fIndexCollisions → global index.
            df_trackextr["fIndexCollisions"] += collision_offset

            # Save results in the container list.
            list_of_df.append( df_trackextr )

            # Update offset for next loop.
            collision_offset += len(df_coll)
```

Cuts from observation of real data

Particle	TPC	$p_{T,daughter}[GeV/c]$	TOF
$K^{\pm}$	$\lvert TPCka \rvert < 2$	[0, 1.4]	$\lvert TOFka \rvert < 4$
		[1.4, 6.0]	$\lvert TOFka \rvert < 3$
		[0, 6.0]	-999
$\pi^{\pm}$	$\lvert TPCpi \rvert < 2$	[0, 1.4]	$\lvert TOFpi \rvert < 4$
		[1.4, 6.0]	$\lvert TOFpi \rvert < 3$
		[0, 6.0]	-999

And on pair variables:

$$\begin{cases} \cos\theta_{pointing} > 0.98 \\ 2GeV/c < p_T < 8GeV/c \\ DcaXY_{product} < -5 \cdot 10^{-5} cm^2 \end{cases}$$

Modeling

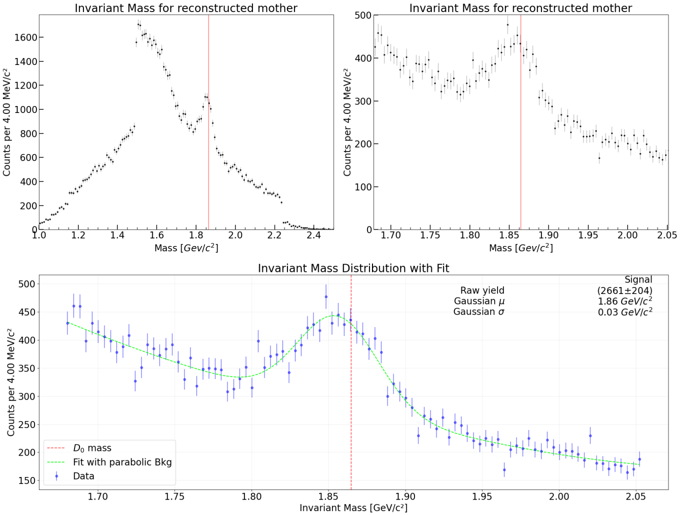
In order to obtain the number of signal particles we interpolate the filtered data. The following assumptions have been used:

$$\text{Fit Function} = k \cdot \left(\frac{N_1}{N_2} \text{Reflected} + \text{Signal} \right) + \text{Background}$$

Where the Reflected component emerges when the wrong assumption on the nature of the particle has been made and, therefore, the invariant mass is incorrect.

- Reflected and Signal are Gaussians ($k \cdot \mathcal{N}(\mu, \sigma^2)$).
- Background is parabolic ($a \cdot x^2 + b \cdot x + c$).
- the ratio N_1/N_2 is fixed from replicating the chosen cuts on MC and taking the ratio $k_{reflected}/k_{signal}$.

Real data observation: results



$2GeV/c < p_T < 8GeV/c$
Raw yield: (2661 ± 204)
Efficiency (from MC): $(1.11 \pm 0.03)\%$

Cuts efficiency

Once we have our cuts we can calculate their efficiency w.r.t. the MC simulation:

$$\textit{Efficiency} = \frac{\#\ \textit{Reconstructed particles}}{\#\ \textit{Generated particles}}$$

Cuts efficiency

Once we have our cuts we can calculate their efficiency w.r.t. the MC simulation:

$$Efficiency = \frac{\#\ Reconstructed\ \ particles}{\#\ Generated\ \ particles}$$

Since the particle counting follow a binomial distribution we can associate to the efficiency the statistical binomial error that for the considered sample is given by:

$$\sigma_{eff} = \sqrt{\frac{N_{reco}}{N_{gen}^2}\left(1 - \frac{N_{reco}}{N_{gen}}\right)}$$

N_{gen}

In the MC simulation data the informations about the generation part are stored in the *O2genparticles* and *O2mccollision* files.
Merging the dataframes on the *fIndexMcCollisions* and selecting:

$$|fPosZ| < 10\ cm$$
$$fPdgCode = \pm 421$$
$$abs(fMainMotherY) < 0.8$$
$$fMainBeautyAncestor = 0$$

N_{gen}

In the MC simulation data the informations about the generation part are stored in the *O2genparticles* and *O2mccollision* files. Merging the dataframes on the *fIndexMcCollisions* and selecting:

$$|fPosZ| < 10 \ cm$$
$$fPdgCode = \pm 421$$
$$abs(fMainMotherY) < 0.8$$
$$fMainBeautyAncestor = 0$$

$\Rightarrow$ 328838 $D_0/\overline{D_0}$ generated with $p_T \in [0, 16)$

N_{reco}

In order to count the reconstructed $D_0/\overline{D_0}$:

N_{reco}

In order to count the reconstructed $D_0/\overline{D_0}$:

1. we carried out the combinatorial computation on the single track informations in the MC files keeping track of the MC information about the real nature of the particles.

N_{reco}

In order to count the reconstructed $D_0/\overline{D_0}$:

1. we carried out the combinatorial computation on the single track informations in the MC files keeping track of the MC information about the real nature of the particles.
2. Apply cuts and selections.

N_{reco}

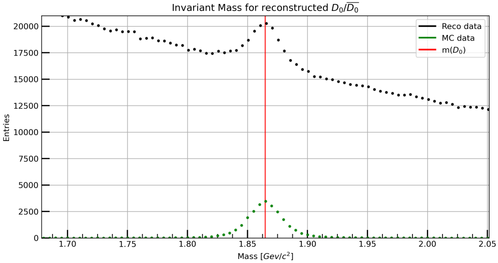
In order to count the reconstructed $D_0/\overline{D_0}$:

1. we carried out the combinatorial computation on the single track informations in the MC files keeping track of the MC information about the real nature of the particles.
2. Apply cuts and selections.
3. Count how many real $D_0/\overline{D_0}$ there are in the remaining data.

N_{reco}

In order to count the reconstructed $D_0/\overline{D_0}$:

1. we carried out the combinatorial computation on the single track informations in the MC files keeping track of the MC information about the real nature of the particles.
2. Apply cuts and selections.
3. Count how many real $D_0/\overline{D_0}$ there are in the remaining data.



Improving the results with MC

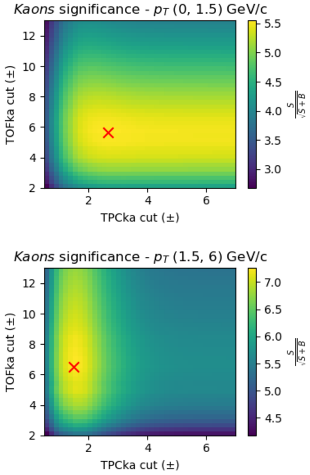
There is a clear peak in the invariant mass distribution of real data, however the efficiency is very low.

$\Downarrow$

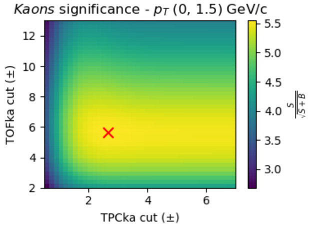
Perform an analysis of single track and pair variables on Monte Carlo and obtain the "best" cuts by maximizing the significance. Given S signal events and B background events, it is defined as:

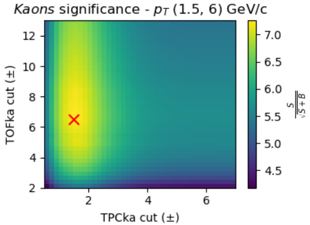
$$Significance = \frac{S}{\sqrt{S+B}}$$

MC: Single track 2D gridsearch



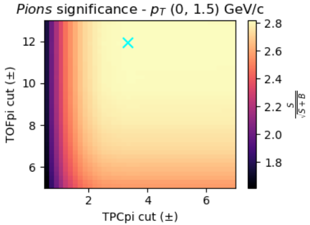
MC: Single track 2D gridsearch

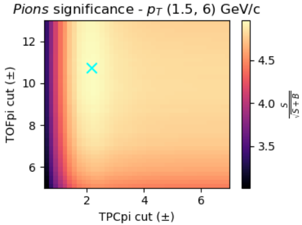




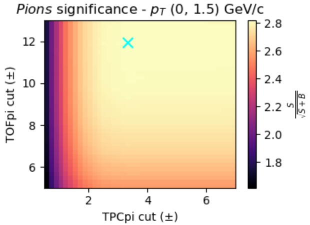
For kaons, well-defined regions of increased significance are visible, indicating that PID cuts are highly effective.
At low p_t, the dominant discriminating variable is TOFka: once TPCka exceeds a threshold of approximately 1–2σ, further tightening of the TPC cut does not lead to a significant improvement in significance.
At higher p_t, their roles are reversed.

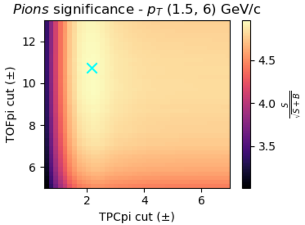
MC: Single track 2D gridsearch





MC: Single track 2D gridsearch

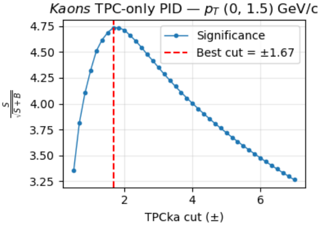


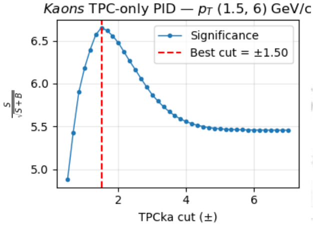


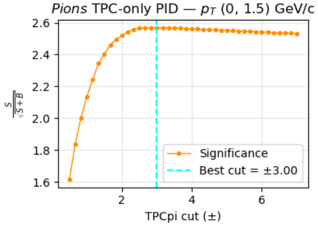
For pions, instead, the significance remains approximately uniform over the entire explored PID cut space. This behavior reflects the intrinsic difficulty of identifying pions originating from D^0 decays, as the background is itself largely composed of pions.
As a consequence, PID cuts on TPC and TOF offer limited discriminating power.

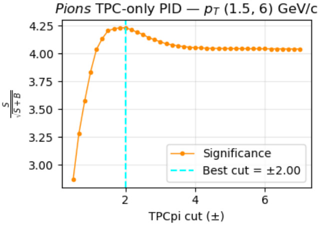
MC: Single track 1D gridsearch

Moreover, for those particles for which the TOF is unavailable (-999), we can optimize the only remaining cut on TPC.









MC: Single track 1D gridsearch

- In the 1D search, the optimal cuts found on the TPC variables are generally consistent with those obtained in the full 2D optimization, but with lower significance. The reduced significance in the TPC-only case reflects the absence of TOF information.
- The most notable difference is observed for kaons at low p_t. This behaviour is consistent with previous search results: TOF provides the dominant discriminating power, and once TOF info is removed, the kaon discrimination is more difficult.
- We tried also using *dcaXY* variable, without success. The distribution of the background and of the signal were very similar.

MC: Single track final cuts

Particle	$p_T[GeV/c]$	TOF	TPC
$K^{\pm}$	[0.0, 1.5]	$\|TOFka\| < 5.67$	$\|TPCka\| < 2.67$
		-999	$\|TPCka\| < 1.67$
	[1.5, 6.0]	$\|TOFka\| < 6.51$	$\|TPCka\| < 1.50$
		-999	$\|TPCka\| < 1.50$
$\pi^{\pm}$	[0.0, 1.5]	$\|TOFpi\| < 11.97$	$\|TPCpi\| < 3.33$
		-999	$\|TPCpi\| < 3.00$
	[1.5, 6.0]	$\|TOFpi\| < 10.74$	$\|TPCpi\| < 2.17$
		-999	$\|TPCpi\| < 2.00$

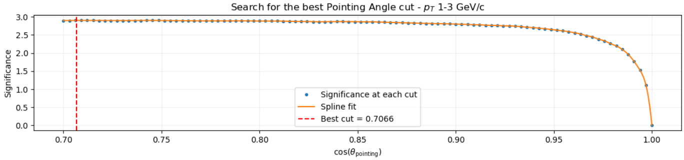
MC: Pair variables gridsearch

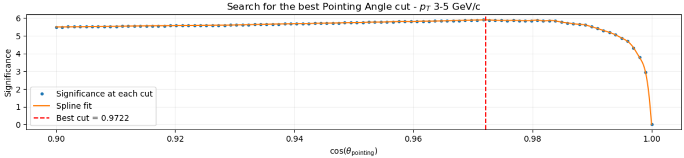
Having found the single tracks best cuts, we can now proceed with the combinatorial part (couples formation).

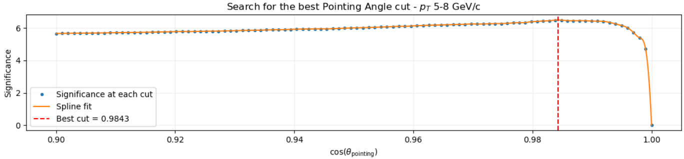
$\Downarrow$

Finally, we can optimize w.r.t. pair variables: $\cos\theta_{pointing}$, $dcaXY_{product}$ and *decay length*.

MC: Pair variables gridsearch





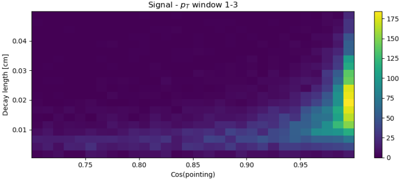


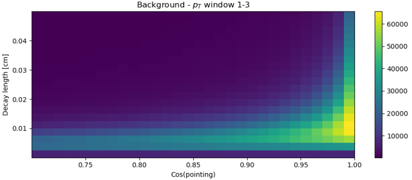
MC: Pair variables 2D gridsearch

We also attempted to optimize cuts by combining the two most promising pair variables, $\cos\theta_{\mathrm{pointing}}$ and *decay length*, in search of potential area cuts.

However, as illustrated in the example below, their distributions are very similar across all p_T ranges. The most significant difference appears to be along the one-dimensional pointing angle cut, which we had already investigated.

Therefore, this combined-variable search did not yield any further improvement.





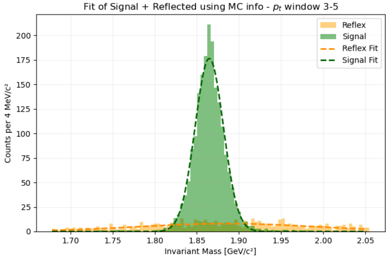
MC: Pair variabs final cuts

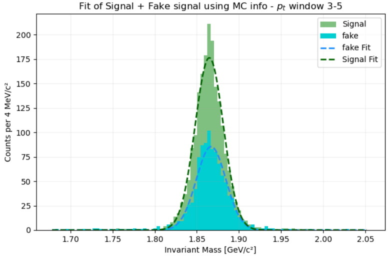
The final cuts we found for pairs variables were:

$$\begin{cases} 1.0\,GeV/c < p_T < 3.0\,GeV/c \\ \cos\theta_{pointing} > 0.71 \\ dcaXY_{prod} < -1.54 \cdot 10^{-5} cm^2 \end{cases} \vee \begin{cases} 3.0\,GeV/c < p_T < 5.0\,GeV/c \\ \cos\theta_{pointing} > 0.97 \\ dcaXY_{prod} < -2.19 \cdot 10^{-5} cm^2 \\ \text{decay length} > 68\mu m \end{cases}$$

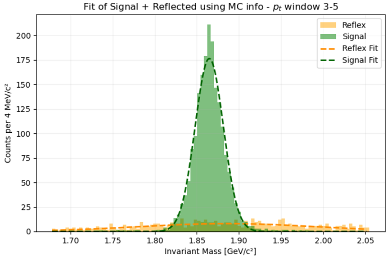
$$\begin{cases} 5.0\,GeV/c < p_T < 8.0\,GeV/c \\ \cos\theta_{pointing} > 0.98 \\ dcaXY_{prod} < -1.75 \cdot 10^{-5} cm^2 \\ \text{decay length} > 116\mu m \end{cases} \vee \begin{cases} 8.0\,GeV/c < p_T < 16.0\,GeV/c \\ \cos\theta_{pointing} > 0.99 \\ dcaXY_{prod} < -0.73 \cdot 10^{-5} cm^2 \\ \text{decay length} > 89\mu m \end{cases}$$

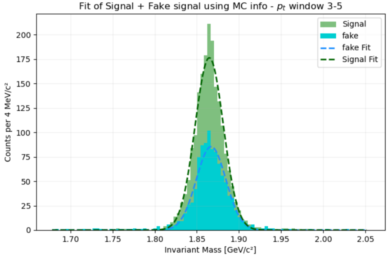
MC: Reflex & beauty components





MC: Reflex & beauty components





At this stage, our dataset contains candidates falling into the D^0, $\overline{D^0}$, or *Both* hypotheses. For the latter case, we retain the invariant masses computed under both the D^0 and $\overline{D^0}$ hypotheses. As a consequence, a **reflection** component is introduced, which must be properly accounted for in the fit.

In addition, a **beauty** component is present. This arises from the fact that the Monte Carlo sample includes D^0 and $\overline{D^0}$ mesons produced in beauty-hadron decays. This contribution must also be explicitly included in the fit model.

MC: fit function

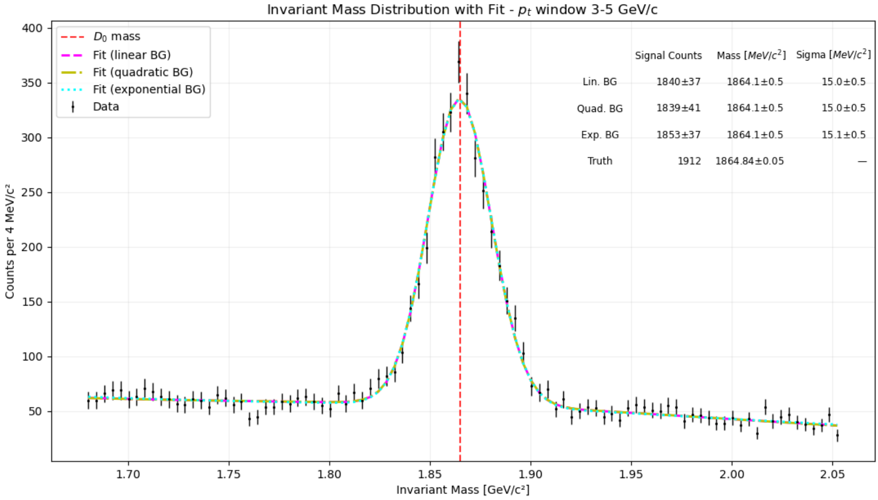
As said, in MC data we also have the beauty component, which means we need to add a gaussian function to our fit function:

$$\text{Fit Function} = k \cdot \left(\frac{N_1}{N_2} \text{Reflex} + \text{Signal} + \frac{N_3}{N_2} \text{Beauty} \right) + \text{Background}$$

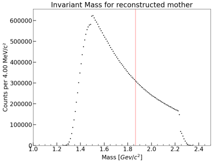
First we obtain the ratios N_1/N_2 and N_3/N_2 respectively from the fits signal-reflex and signal-beauty. In this way, k is the only free parameter.
Then we proceed with the fit of each p_t window using all of these three possible background functions: linear, quadratic, exponential.

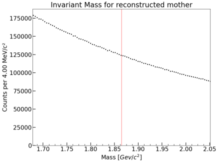
MC: results example

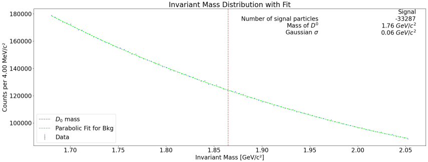
For $3.0 GeV/c < p_T < 5.0 GeV/c$ and the best pair variable cuts:



From MC to real data

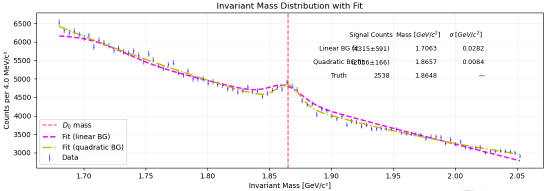




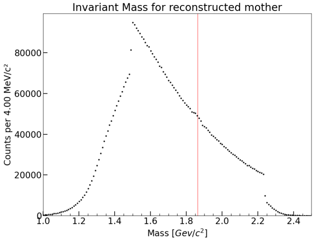


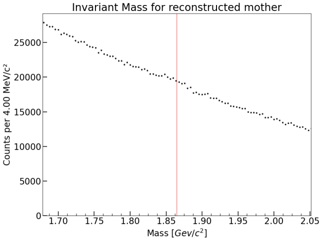
- $0.0 GeV/c < p_T < 1.0 GeV/c$
- Efficiency: $(3.68 \pm 0.07)\%$
- MC values don't work well in this interval even though they do in the simulation.

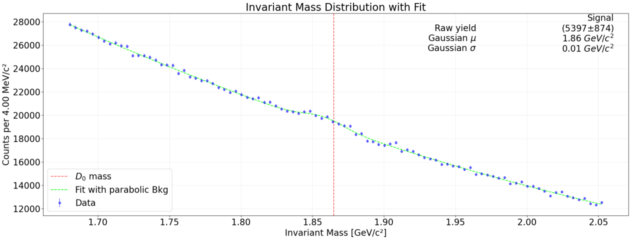
$\Downarrow$(MC)



From MC to real data





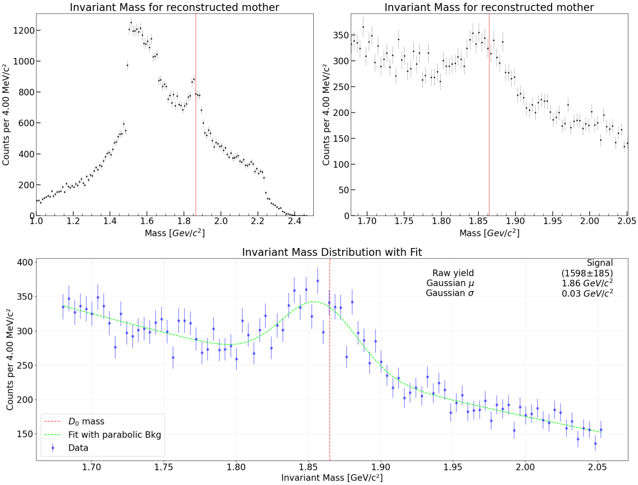


$$1.0 GeV/c < p_T < 3.0 GeV/c$$

Raw yield: (5397 ± 874)

Efficiency (from MC): $(3.16 \pm 0.04)\%$

From MC to real data

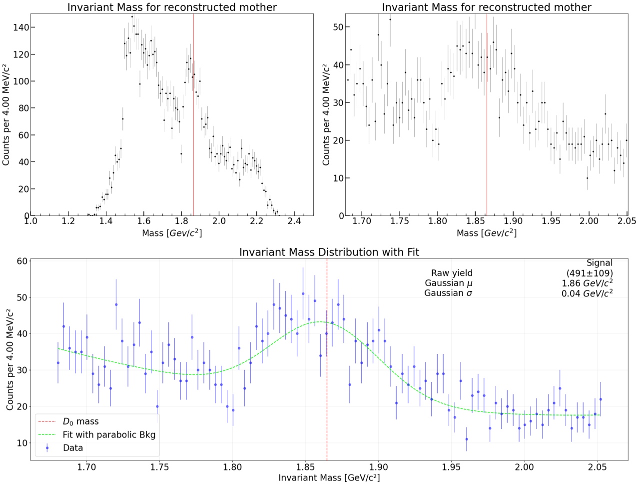


$3.0 GeV/c < p_T < 5.0 GeV/c$

Raw yield: (1598 ± 185)

Efficiency (from MC): $(3.59 \pm 0.08)\%$

From MC to real data

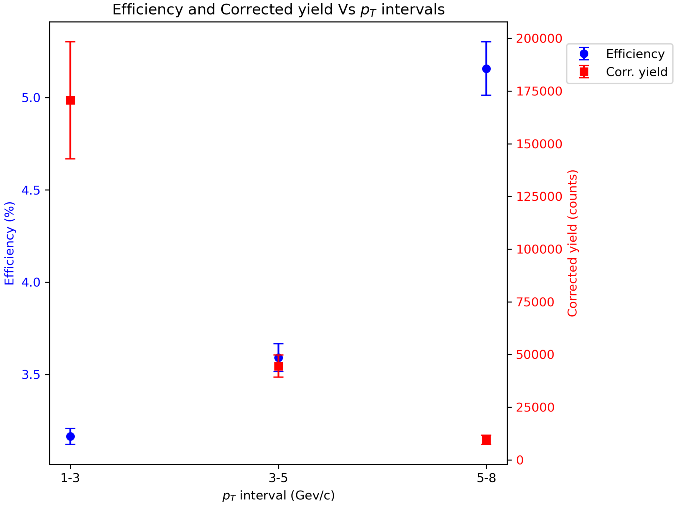


$$5.0 GeV/c < p_T < 8.0 GeV/c$$

Raw yield: (491 ± 109)

Efficiency (from MC): $(5.2 \pm 0.1)\%$

Efficiency and Corrected yield



$$Corr.yield = raw\ \ yield/efficiency$$

"Classical" analysis conclusions

- $D^0 \to K^-\pi^+$ found with both direct study of available variables and with MC simulation.
- Results on MC do not appear to be fully replicable on the real dataset (possibly problem with the TOF; also beauty abundance is not the same as in nature.)
- Still, having MC as a tool is greatly useful as it allows to evaluate the "goodness" of filters via significance or efficiency.
- MC cuts, overall, yield better results (e.g. better efficiency) and greatly diminish the reflected component.

Machine Learning approach

Manually searching for the optimal cuts is inefficient, even more for large datasets.
This becomes even more evident searching for Λ_c^+ decay events, that are more rare and need three different particles.
The next part of the analysis takes into consideration:

- Candidate files, the analysis is done starting from files of preselected particles.
- Machine Learning Classifier, the particles are sorted into a multi-class classification to determine their fitting score.
- Grid Search Training, to find the optimal set of cuts to enhance the significance of the dataset.
- Selected candidates, the candidates that pass the cut selection are then analysed as in the 'classical' approach.

Candidate files

The combinatorial problem presented before becomes worse taking into account a larger number of collisions and the possibility of a three particles decay.
For the same collective data acquisition we are provided with:

Candidate files

The combinatorial problem presented before becomes worse taking into account a larger number of collisions and the possibility of a three particles decay.

For the same collective data acquisition we are provided with:

- 62 Data Chunks of Candidates for both D_0 and Λ_c^+.
- Each data chunks is a data frame with single particle (daughters) and combinatorial (mother) properties for each entry.
- The total size of the data is $\approx 60\,GB$ of TTree Files.

Candidate files

The combinatorial problem presented before becomes worse taking into account a larger number of collisions and the possibility of a three particles decay.

For the same collective data acquisition we are provided with:

- 62 Data Chunks of Candidates for both D_0 and Λ_c^+.
- Each data chunks is a data frame with single particle (daughters) and combinatorial (mother) properties for each entry.
- The total size of the data is $\approx 60\,GB$ of TTree Files.
- Data are correlated with Monte Carlo simulations files, where only D_0 and $\bar{D}_0$ or Λ_c^+ are selected.
- The integrated Number of Events and Luminosity for the whole data acquisition is provided in a TH1F object.

ML: Dataset subdivision

To better handle the large data size and find the optimal parameters for each energy region, the analysis has been split into separated subset of transverse momentum of the mother particle (the candidate):

D_0	$(0; 2)$	$(2; 3.2)$	$(3.2; 5)$	$(5; 20)$
Λ_c^+	$(0; 3.2)$	-	$(3.2; 5)$	$(5; 20)$

Table: p_T $[GeV/c]$ ranges for the preselection and training

This is an approximative division made in order to distribute mostly uniformly the data content during the analysis.

ML: Training Dataset

Monte Carlo Candidate files contain only the desired particles. In order to train the algorithm we have created, for each p_T interval:

ML: Training Dataset

Monte Carlo Candidate files contain only the desired particles. In order to train the algorithm we have created, for each p_T interval:

- Training Set, a dataset of $\approx 30k$ particles; $\approx 20k$ are sampled randomly from the MC dataset, half prompt signal, half beauty signal;

ML: Training Dataset

Monte Carlo Candidate files contain only the desired particles. In order to train the algorithm we have created, for each p_T interval:

- Training Set, a dataset of $\approx 30k$ particles; $\approx 20k$ are sampled randomly from the MC dataset, half prompt signal, half beauty signal;
- The remaining $\approx 10k$ candidates are randomly sampled from the "tail" regions of the first data chunk (01).
- In order to do so, from the MC data we extracted the peak region in the invariant mass domain, the one containing almost all signal particles, and the remaining region is considered the "tail" region.

ML: Training Dataset

Monte Carlo Candidate files contain only the desired particles. In order to train the algorithm we have created, for each p_T interval:

- Training Set, a dataset of $\approx 30k$ particles; $\approx 20k$ are sampled randomly from the MC dataset, half prompt signal, half beauty signal;
- The remaining $\approx 10k$ candidates are randomly sampled from the "tail" regions of the first data chunk (01).
- In order to do so, from the MC data we extracted the peak region in the invariant mass domain, the one containing almost all signal particles, and the remaining region is considered the "tail" region.
- From the MC data analysis some preliminary mass cuts have been implemented to center the data around the peak
- The other part of unused MC data have been used as Test set, and used to compute the efficiency.

ML: Training Dataset

Example of a mass peak region and tails; inside the red lines is considered peak region, between red and blue tail and outside blue lines data are discarded.

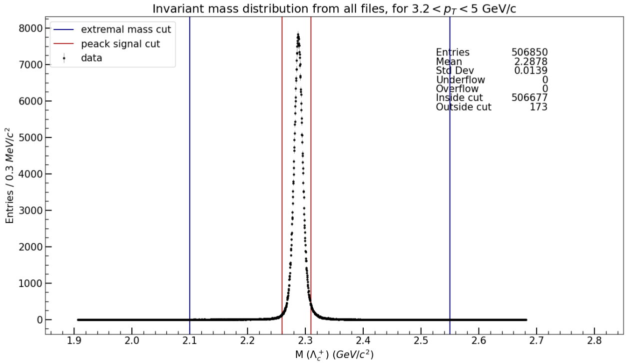


Figure: Λ_c^+ Invariant Mass plot from MC data in the indicated p_T region

ML: Dataset Preselection

After the initial test we found that the algorithm struggle to detect the Reflected component originated from D_0 - $\bar{D}_0$ identification.

ML: Dataset Preselection

After the initial test we found that the algorithm struggle to detect the Reflected component originated from D_0 - $\bar{D}_0$ identification. Using a set of preselection cuts over the single particle properties (mainly $\sigma(TPC)$ for each species) studied in MC properties we reduced that component enough to be integrated into the background fit.

ML: Dataset Preselection

After the initial test we found that the algorithm struggle to detect the Reflected component originated from D_0 - $\bar{D}_0$ identification.
Using a set of preselection cuts over the single particle properties (mainly $\sigma(TPC)$ for each species) studied in MC properties we reduced that component enough to be integrated into the background fit.
In the training the Reflected component was removed to improve the results.

Mother Particle	Particle	TPC
D_0	$K^{\pm}$	$\lvert TPCka\rvert < 2$
	$\pi^{\pm}$	$\lvert TPCpi\rvert < 2$
Λ_c^+	$K^{\pm}$	$\lvert TPCka\rvert < 2$
	$\pi^{\pm}$	$\lvert TPCpi\rvert < 2$
	p	$\lvert TPCpr\rvert < 2$

ML: Dataset Preselection

Example of a the preselection cut effect on a randomly sampled dataset made of Monte Carlo signal and Data noise.

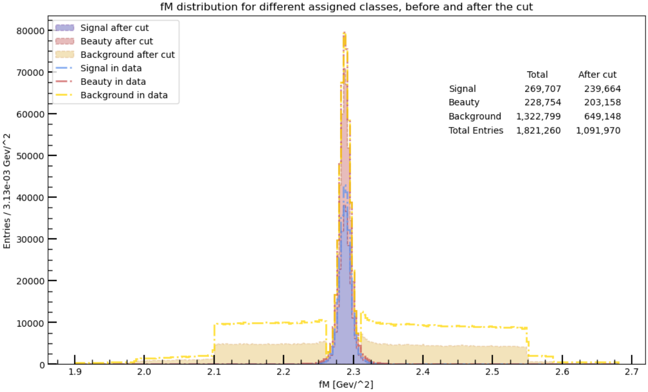


Figure: Preliminary cuts on a Λ_c^+ dataset in [3.2, 5] GeV/c p_T region

ML: Dataset Preselection

Here we can observe how much the reflected region is reduced and flattened.

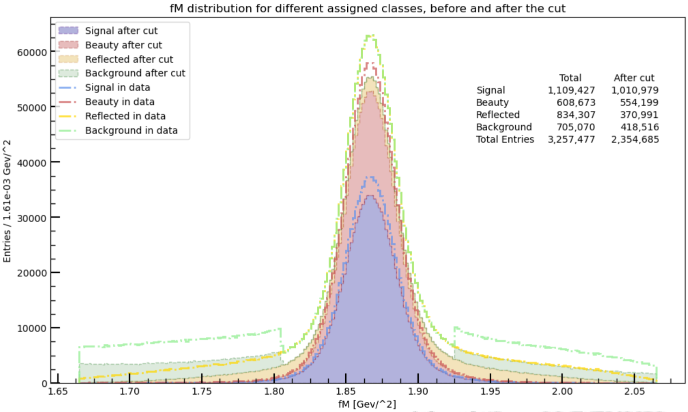


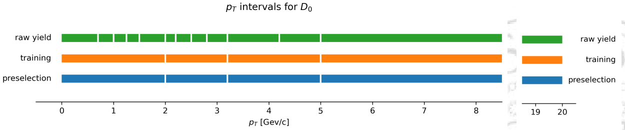
Figure: Preliminary cuts on a D_0 dataset in [2.0, 3.2] GeV/c p_T region

ML: Dataset subdivision

For the optimal cut search and the raw yield extraction, we have divided each of the previous momentum set into more subsets:

ML: Dataset subdivision

For the optimal cut search and the raw yield extraction, we have divided each of the previous momentum set into more subsets:



ML: Dataset subdivision

For the optimal cut search and the raw yield extraction, we have divided each of the previous momentum set into more subsets:

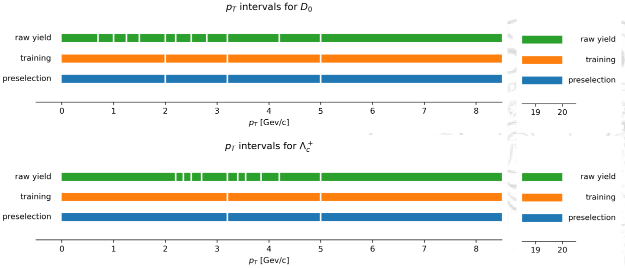


Figure: D_0 and Λ_c^+ subdivision into p_T regions

ML: Algorithm

To classify the particles we used a XGBoost Classifier (Extreme Gradient Boosting) to perform a supervised multi class training. The training dataset is divided into 3 classes:

ML: Algorithm

To classify the particles we used a XGBoost Classifier (Extreme Gradient Boosting) to perform a supervised multi class training. The training dataset is divided into 3 classes:

- Prompt signal, the MC particles identified as the properly selected particle.

ML: Algorithm

To classify the particles we used a XGBoost Classifier (Extreme Gradient Boosting) to perform a supervised multi class training. The training dataset is divided into 3 classes:

- Prompt signal, the MC particles identified as the properly selected particle.
- Beauty signal, particles that match with the desired species but comes from decays containing the quark beauty, so they cannot be used in the quark charm production analysis.

ML: Algorithm

To classify the particles we used a XGBoost Classifier (Extreme Gradient Boosting) to perform a supervised multi class training. The training dataset is divided into 3 classes:

- Prompt signal, the MC particles identified as the properly selected particle.
- Beauty signal, particles that match with the desired species but comes from decays containing the quark beauty, so they cannot be used in the quark charm production analysis.
- Background, particles that are not identified as the selected signal.

ML: Algorithm

To classify the particles we used a XGBoost Classifier (Extreme Gradient Boosting) to perform a supervised multi class training. The training dataset is divided into 3 classes:

- Prompt signal, the MC particles identified as the properly selected particle.
- Beauty signal, particles that match with the desired species but comes from decays containing the quark beauty, so they cannot be used in the quark charm production analysis.
- Background, particles that are not identified as the selected signal.

Once the parameter of the classifier are optimized for the **softmax** objective, the model is trained and is able to output the **probability** for each candidate to fit in each class.
The model uses a number of simpler sub classifiers that are combined to obtain a smart and efficient global classifier.

ML: Algorithm procedure

The steps the whole algorithm takse are:

ML: Algorithm procedure

The steps the whole algorithm takse are:

- Feature definition, a user inserted list of properties to use in the training.

ML: Algorithm procedure

The steps the whole algorithm takse are:

- Feature definition, a user inserted list of properties to use in the training.
- Hyper Parameter Optimization, automatic grid search over the model hyper parameters to find the optimal configuration for the multi classification problem of this dataset;

ML: Algorithm procedure

The steps the whole algorithm takse are:

- Feature definition, a user inserted list of properties to use in the training.
- Hyper Parameter Optimization, automatic grid search over the model hyper parameters to find the optimal configuration for the multi classification problem of this dataset;
- KFold Training, the Training Dataset is divided into K (10) subset, cycling over all 10 subsets as validation set while using the remaining 9 as training set to find the best model without needing more points;

ML: Algorithm procedure

The steps the whole algorithm takse are:

- Feature definition, a user inserted list of properties to use in the training.
- Hyper Parameter Optimization, automatic grid search over the model hyper parameters to find the optimal configuration for the multi classification problem of this dataset;
- KFold Training, the Training Dataset is divided into K (10) subset, cycling over all 10 subsets as validation set while using the remaining 9 as training set to find the best model without needing more points;
- Result Evaluation, returning a series of different parameters to evaluate the goodness of the fit (confusion matrix, importance graph, beeswarm feature graph, multiple ROC curves).

ML: Features

The set of features fond to work best in this environment are:

- **fChi2PCA**, the estimator for the goodness of the secondary vertex
- **fDecayLength**, the estimated decay length
- **fDecayLengthXY**, the decay length projection on XY plane
- **fImpactParameter0**, the DCA on XY plane to the primary vertex for the first particle
- **fImpactParameter1**, the DCA on XY plane to the primary vertex for the second particle
- **fImpactParameter2**, the DCA on XY plane to the primary vertex for the third particle (Λ_c^+ only)
- **fImpactParameterProduct**, the product of DCA for first and second particle (D_0 only)
- **fCpa**, cosine of pointing angle
- **fCpaXY**, cosine of pointing angle projected on XY plane
- **fCt**, estimated life time path (Λ_c^+ only)

ML: Evaluation metrics

We can provide some examples of evaluation metrics obtained for the training

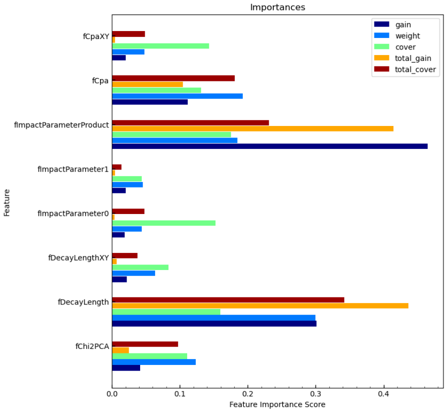


Figure: Feature importance for D_0 training in [0.0, 2.0] GeV/c p_T region

ML: Evaluation metrics

In multi classification problems, many roc curves can be computed, these are the 'one vs the rest' for all 3 classes

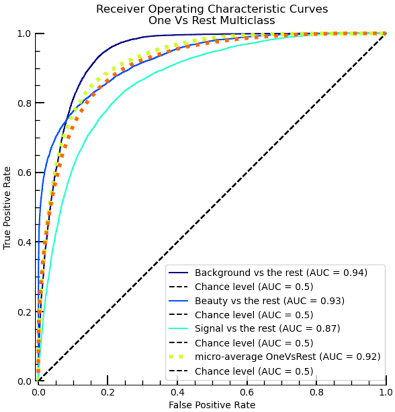


Figure: ROC curves for D_0 training in [3.2, 5] GeV/c p_T region

ML: Evaluation metrics

The beeswarm (SHAP) plot is a good tool to visualize the impact of each feature toward a class selection. This can be done for each class and has been used to define the definitive features.

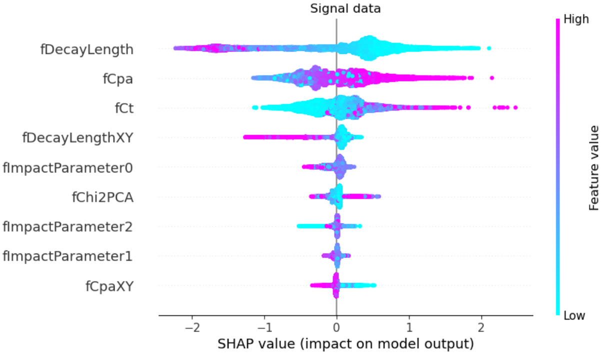


Figure: Beeswarm plot for (prompt) signal class in Λ_c^+ training in [0, 3.2] GeV/c p_T region

ML: Class Prediction

Once the model is trained, it is able to assign to each candidate the probability of belonging to each class.

ML: Class Prediction

Once the model is trained, it is able to assign to each candidate the probability of belonging to each class.
Using this model directly on the test set gives us

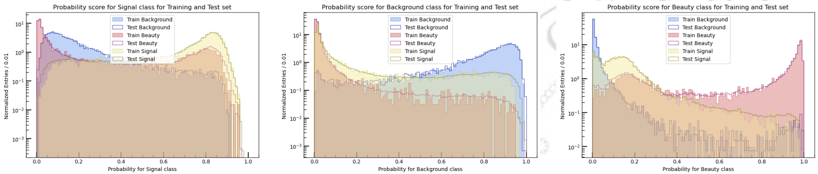


Figure: Class prediction probabilities histogram for D_0 training in [5, 20] GeV/c p_T region, from left to right, the probability for each candidate to be classified as Prompt Signal, Background or Beauty Signal

ML: Class Probability Cuts

For each probability we can obtain the probability of being labelled as Signal or Background class.
We can fix a threshold on the signal probability and accept only those.

ML: Class Probability Cuts

For each probability we can obtain the probability of being labelled as Signal or Background class.
We can fix a threshold on the signal probability and accept only those.
But we can do better: we can find the best threshold over all class probabilities to maximize the signal in set of selected candidates.

ML: Class Probability Cuts

For each probability we can obtain the probability of being labelled as Signal or Background class.
We can fix a threshold on the signal probability and accept only those.
But we can do better: we can find the best threshold over all class probabilities to maximize the signal in set of selected candidates.
To do this we implement a grid search using as metric the (pseudo) significance of the resulting set (given Monte Carlo information of real signal)

$$S = \frac{signal}{\sqrt{signal + background}} \approx \frac{signal}{\sqrt{background}}$$

ML: Probability Cuts grid search

We can find for each subset the optimal threshold set:

- New Training Set, extracting a small subset from the Test set for this "second training"
- Automatic grid search, using (pseudo)significance as evaluating method

ML: Probability Cuts grid search

We can find for each subset the optimal threshold set:

- New Training Set, extracting a small subset from the Test set for this "second training"
- Automatic grid search, using (pseudo)significance as evaluating method

For the convention used, a candidate is **accepted** if it has:

- Prompt **Signal** probability $>$ signal threshold
- **Beauty** Signal probability $<$ beauty threshold
- **Background** probability $<$ background threshold

ML: Probability Cuts grid search

For each training subrange the search is run producing a similar graph over N=500 trials

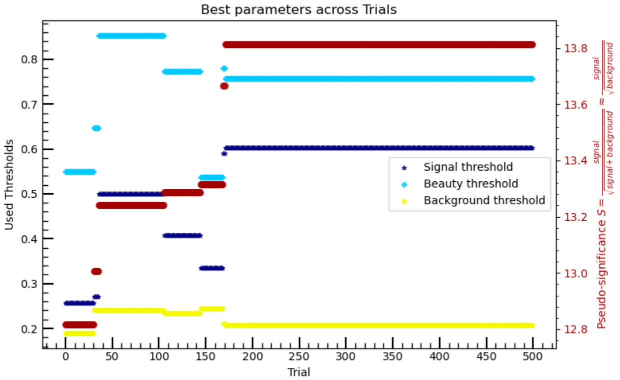


Figure: Best parameters for Λ_c^+ probabilities thresholds in [2.5, 2.7] GeV/c p_T region, in red the Pseudo-Significance

ML: Model application

Using the trained model with the probability threshold cuts found, we can compute the efficiency for each subrange.
We can also compute the beauty signal efficiency, to check if it remains marginal.

ML: Model application

Using the trained model with the probability threshold cuts found, we can compute the efficiency for each subrange.
We can also compute the beauty signal efficiency, to check if it remains marginal.
To compute the **efficiency** we use the remaining Monte Carlo dataset (used only as test set) divided into momentum subranges.

ML: Model application

Using the trained model with the probability threshold cuts found, we can compute the efficiency for each subrange.
We can also compute the beauty signal efficiency, to check if it remains marginal.
To compute the **efficiency** we use the remaining Monte Carlo dataset (used only as test set) divided into momentum subranges.
Using the trained model over the data chunks divided into momentum subranges we are able to extract new datasets of **selected candidates**.

ML: Model application

Using the trained model with the probability threshold cuts found, we can compute the efficiency for each subrange.
We can also compute the beauty signal efficiency, to check if it remains marginal.
To compute the **efficiency** we use the remaining Monte Carlo dataset (used only as test set) divided into momentum subranges.
Using the trained model over the data chunks divided into momentum subranges we are able to extract new datasets of **selected candidates**. Over the selected dataset we can reproduce the already used **interpolation** procedure to extract the **raw yield**.

ML: Model application

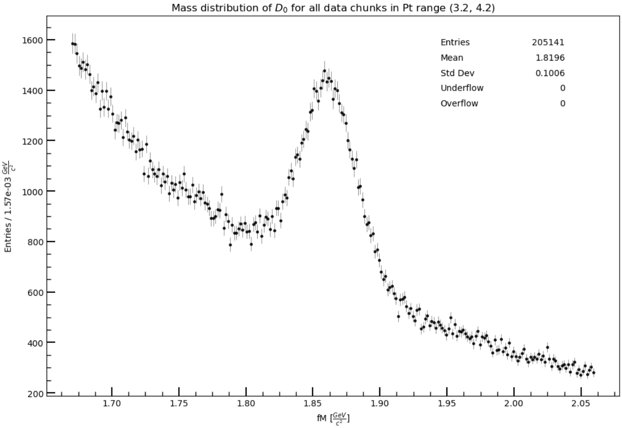


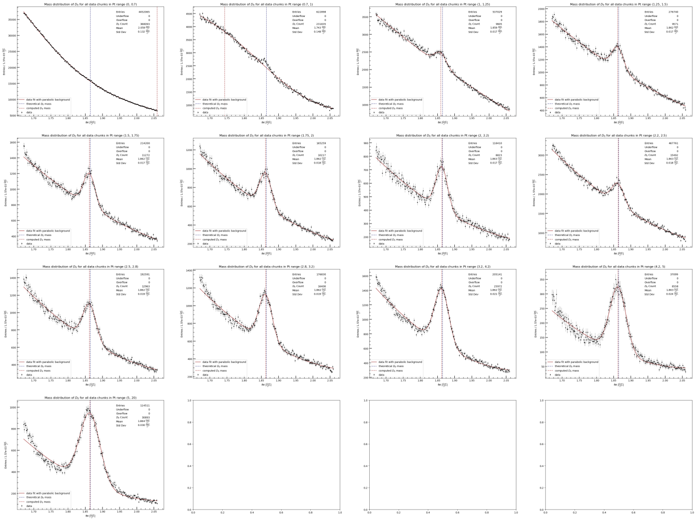
Figure: Selected dataset of D_0 candidates in [3.2, 4.2] GeV/c p_T subregion, obtained adding all 61 chunks selected candidates

ML: Raw Yield extraction

Using a parabolic background and a Gaussian function for the peak we are able to extract the raw yield for each momentum subset. For each interpolation is also possible to get an estimate of the particle mass, compute the pull with respect to the PDG value and use this as and indicator of which region to not be considered in further analysis.

$$pull = \frac{|m_{computed} - m_{PDG}|}{\sqrt{\sigma^2_{computed} + \sigma^2_{PDG}}} \approx \frac{|m_{computed} - m_{PDG}|}{\sigma_{computed}}$$

ML: D_0 Raw Yield



ML: D_0 Summary

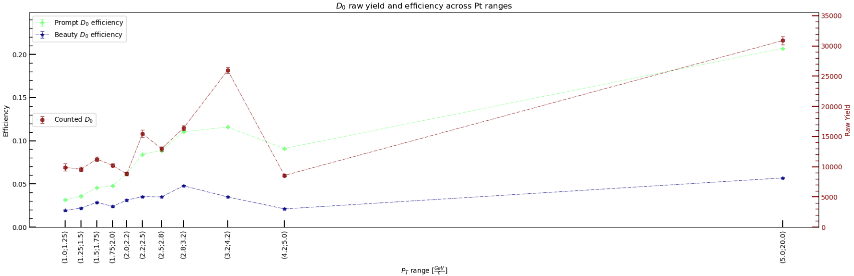
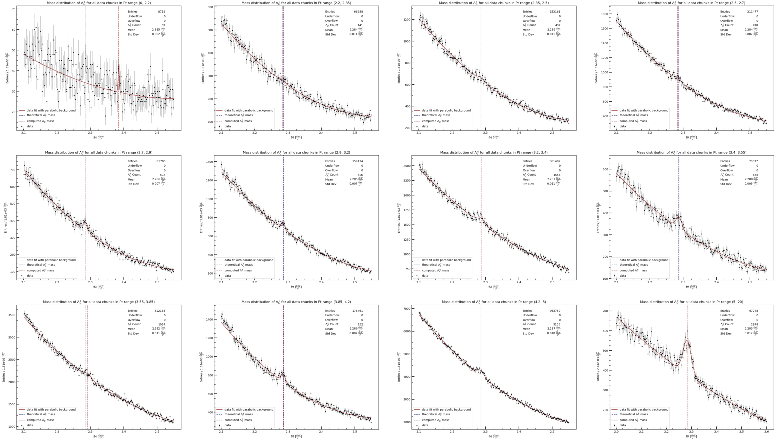


Figure: Summary graph for efficiency and raw yield for D_0

ML: Λ_c^+ Raw Yield



ML: Λ_c^+ Summary

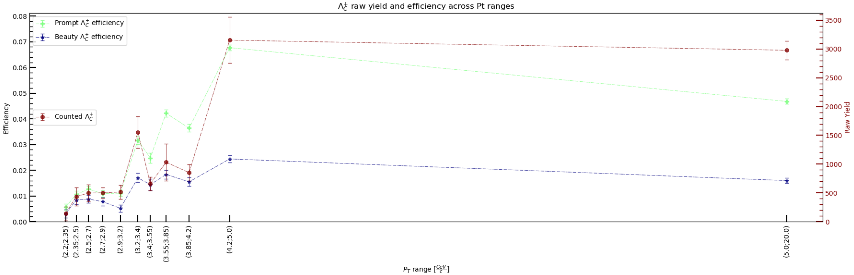


Figure: Summary graph for efficiency and raw yield for Λ_c^+

Candidate: Yield and Ratio

We can now compute the corrected yield for each p_T range

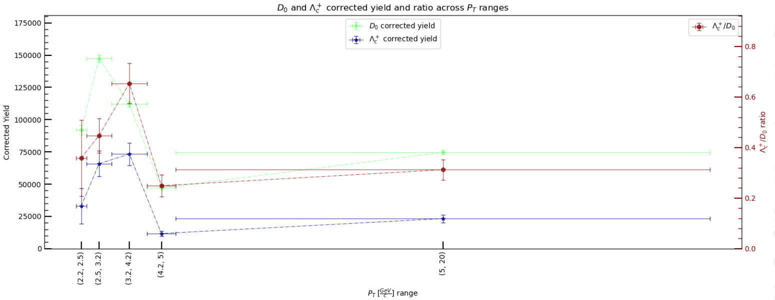
$$\mathit{corrected\ yield} = \frac{\mathit{raw\ yield}}{\mathit{efficiency}}$$

Candidate: Yield and Ratio

We can now compute the corrected yield for each p_T range

$$\textit{corrected yield} = \frac{\textit{raw yield}}{\textit{efficiency}}$$

From the corrected yield, for the matching total p_T ranges we can compute the ratio of particle produced Λ_c^+/D_0



Production rate and Cross section

Using the system parameter to integrate the luminosity and particle production for the used data chunks we can compute the differential cross section $\sigma = \frac{yield}{2\mathcal{L}}$ and production ratio $R = \frac{yield}{2N}$, for luminosity $\mathcal{L}$, N total events and correction factor of 2.

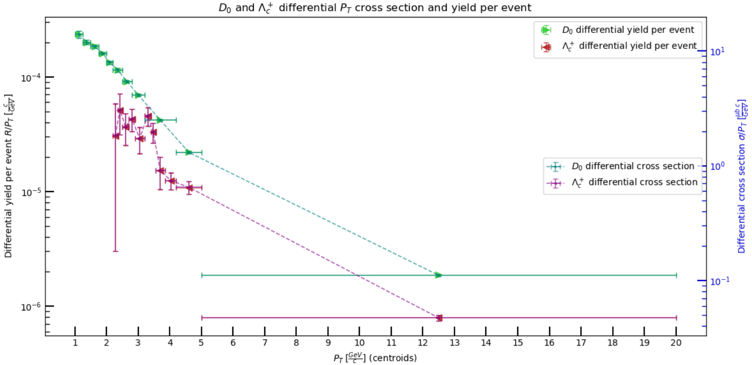


Figure: D_0 and Λ_c^+ $\frac{d\sigma}{dp_T}$ and $\frac{dR}{dp_T}$, similar point with different scales

ML: Conclusion

Comparing the previous results for Λ_c^+/D_0 ratio with what we found

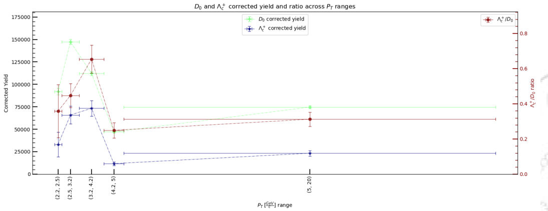


Figure: Our results

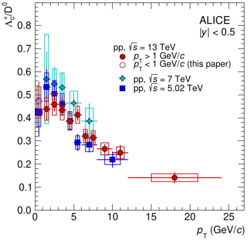


Figure: ALICE results[3]

Errors reported in left graph are only systematic (underestimation).

[3] "Charm production and fragmentation fractions at midrapidity in pp collisions at $\sqrt{s} = 13$ TeV", 2308.04877, ALICE

Appendix: Dataset p_T ranges

D_0 Range	p_T $[GeV/c]$ Sub-ranges	Λ_c^+ Range	p_T $[GeV/c]$ Sub-ranges
(0; 2)	(0; 0.7), (0.7; 1), (1; 1.25), (1.25; 1.5), (1.5; 1.75), (1.75; 2)	(0; 3.2)	(0; 2.2),
(2; 3.2)	(2; 2.2), (2.2; 2.5), (2.5; 2.8), (2.8; 3.2)		(2.2; 2.35), (2.35; 2.5), (2.5; 2.7), (2.7; 2.9), (2.9; 3.2)
(3.2; 5)	(3.2; 4.2), (4.2; 5)	(3.2; 5)	(3.2; 3.4), (3.4; 3.55), (3.55; 3.85), (3.85; 4.2), (4.2; 5)
(5; 20)	(5; 20)	(5; 20)	(5; 20)

Table: p_T sub ranges for cut optimization and raw yield extraction

Appendix: Dataset Preselection

Other preselection cuts on daughter particle properties have been made to improve the Signal/Background ratio.

Mother Particle	$p_T[GeV/c]$	Property cut
	[0.0, 2.0]	$0.5 < fPtProng0 < 2.0$ $0.5 < fPtProng1 < 2.0$
D_0	[2.0, 3.2]	$0.5 < fPtProng0 < 3.0$ $0.5 < fPtProng1 < 3.0$
	[3.2, 5]	$0.5 < fPtProng0 < 4.0$ $0.5 < fPtProng1 < 4.0$
	[5, 20]	$0.5 < fPtProng0 < 10.0$ $0.5 < fPtProng1 < 10.0$

Appendix: Dataset Preselection

Mother Particle	$p_T[GeV/c]$	Property cut
Λ_c^+	[0.0, 3.2]	$fPtProng0 < 2.0$ $fPtProng1 < 1.75$ $fPtProng2 < 1.5$
	[3.2, 5]	$fPtProng0 < 3.5$ $fPtProng1 < 3.0$ $fPtProng2 < 2.75$
	[5, 20]	$fPtProng0 < 6.0$ $fPtProng1 < 6.0$ $fPtProng2 < 6.0$

Appendix: D_0 counting

p_T range $[GeV/c]$	Raw Yield	Corrected Yield
(0.0, 0.7)	368093±1193243	17281362±56022915
(0.7, 1.0)	231835±379317	9462653±15484061
(1.0, 1.25)	9905±615	313449±20664
(1.25, 1.5)	9571±359	267346±11309
(1.5, 1.75)	11272±336	246114±8244
(1.75, 2.0)	10217±267	214193±6420
(2.0, 2.2)	8815±273	142177±4863
(2.2, 2.5)	15492±579	184209±7053
(2.5, 2.8)	12963±362	145980±4283
(2.8, 3.2)	16408±420	148758±3923
(3.2, 4.2)	25972±497	224283±4446
(4.2, 5.0)	8558±194	94044±2326
(5.0, 20.0)	30893±645	149314±3147

Appendix: D_0 Properties

p_T range $[GeV/c]$	Prompt Efficiency	Beauty Efficiency	Pull
(0.0, 0.7)	2.13%±0.06%	2.09%±0.08%	8.45
(0.7, 1.0)	2.45%±0.06%	1.97%±0.08%	4.98
(1.0, 1.25)	3.16%±0.07%	1.93%±0.09%	0.65
(1.25, 1.5)	3.58%±0.07%	2.20%±0.09%	0.34
(1.5, 1.75)	4.58%±0.07%	2.86%±0.09%	0.35
(1.75, 2.0)	4.77%±0.07%	2.40%±0.09%	0.34
(2.0, 2.2)	6.20%±0.09%	3.12%±0.11%	0.28
(2.2, 2.5)	8.41%±0.07%	3.53%±0.09%	0.23
(2.5, 2.8)	8.88%±0.08%	3.49%±0.10%	0.28
(2.8, 3.2)	11.03%±0.07%	4.78%±0.09%	0.29
(3.2, 4.2)	11.58%±0.06%	3.49%±0.07%	0.29
(4.2, 5.0)	9.10%±0.09%	2.11%±0.10%	0.21
(5.0, 20.0)	20.69%±0.06%	5.69%±0.07%	0.11

Appendix: Λ_c^+ counting

p_T range $[GeV/c]$	Raw Yield	Corrected Yield
(0.0, 2.2)	32±14	21333±10262
(2.2, 2.35)	141±123	24737±22304
(2.35, 2.5)	437±161	41226±16008
(2.5, 2.7)	498±145	39213±12004
(2.7, 2.9)	502±93	46055±10147
(2.9, 3.2)	516±120	46486±11922
(3.2, 3.4)	1556±277	49085±9042
(3.4, 3.55)	658±117	26532±5137
(3.55, 3.85)	1034±324	24502±7721
(3.85, 4.2)	852±142	23407±4019
(4.2, 5.0)	3155±402	46603±5995
(5.0, 20.0)	2978±161	63632±3699

Appendix: Λ_c^+ Properties

p_T range $[GeV/c]$	Prompt Efficiency	Beauty Efficiency	Pull
(0.0, 2.2)	0.15%±0.03%	0.06%±0.04%	38.07
(2.2, 2.35)	0.57%±0.13%	0.31%±0.16%	0.34
(2.35, 2.5)	1.06%±0.13%	0.84%±0.17%	0.01
(2.5, 2.7)	1.27%±0.12%	0.88%±0.15%	0.47
(2.7, 2.9)	1.09%±0.13%	0.78%±0.16%	0.01
(2.9, 3.2)	1.11%±0.12%	0.52%±0.14%	0.23
(3.2, 3.4)	3.17%±0.15%	1.71%±0.18%	0.05
(3.4, 3.55)	2.48%±0.19%	1.44%±0.22%	0.22
(3.55, 3.85)	4.22%±0.14%	1.84%±0.17%	0.82
(3.85, 4.2)	3.64%±0.15%	1.55%±0.17%	0.12
(4.2, 5.0)	6.77%±0.12%	2.44%±0.14%	0.03
(5.0, 20.0)	4.68%±0.10%	1.60%±0.11%	0.39

Appendix: Λ_c^+/D_0 ratio

p_T range $[GeV/c]$	Ratio Λ_c^+/D_0
(2.2, 2.5)	0.36±0.15
(2.5, 3.2)	0.45±0.07
(3.2, 4.2)	0.65±0.08
(4.2, 5)	0.25±0.04
(5, 20)	0.31±0.04

Appendix: D_0 Final results

p_T range [GeV/c]	Diff Yield/Event [c/GeV]	$\frac{d\sigma_{CS}}{dp_T}$ [$\mu b\ c/GeV$]	Corrected $\frac{d\sigma_{CS}}{dp_T}$ [$\mu b\ c/GeV$]
(1.0, 1.25)	2.35e-04±2e-05	13.967±0.921	353.9
(1.25, 1.5)	2.01e-04±8e-06	11.912±0.504	301.8
(1.5, 1.75)	1.85e-04±6e-06	10.966±0.367	277.8
(1.75, 2.0)	1.61e-04±5e-06	9.544±0.286	241.8
(2.0, 2.2)	1.33e-04±5e-06	7.919±0.271	200.6
(2.2, 2.5)	1.15e-04±4e-06	6.840±0.262	173.3
(2.5, 2.8)	9.13e-05±3e-06	5.420±0.159	137.3
(2.8, 3.2)	6.97e-05±2e-06	4.143±0.109	105.0
(3.2, 4.2)	4.21e-05±8e-07	2.498±0.050	63.3
(4.2, 5.0)	2.20e-05±5e-07	1.310±0.032	33.2
(5.0, 20.0)	1.87e-06±4e-08	0.111±0.002	2.8

Appendix: Λ_c^+ Final results

p_T range [GeV/c]	Diff Yield/Event [c/GeV]	$\frac{d\sigma_{CS}}{dp_T}$ [$\mu b\ c/GeV$]	Corrected $\frac{d\sigma_{CS}}{dp_T}$ [$\mu b\ c/GeV$]
(2.2, 2.35)	3.07e-05$\pm$3e-05	1.837$\pm$1.656	29.49
(2.35, 2.5)	5.11e-05$\pm$2e-05	3.062$\pm$1.189	49.14
(2.5, 2.7)	3.65e-05$\pm$1e-05	2.184$\pm$0.669	35.06
(2.7, 2.9)	4.28e-05$\pm$9e-06	2.565$\pm$0.565	41.17
(2.9, 3.2)	2.88e-05$\pm$7e-06	1.726$\pm$0.443	27.71
(3.2, 3.4)	4.57e-05$\pm$8e-06	2.734$\pm$0.504	43.88
(3.4, 3.55)	3.29e-05$\pm$6e-06	1.970$\pm$0.381	31.63
(3.55, 3.85)	1.52e-05$\pm$5e-06	0.910$\pm$0.287	14.60
(3.85, 4.2)	1.24e-05$\pm$2e-06	0.745$\pm$0.128	11.96
(4.2, 5.0)	1.08e-05$\pm$1e-06	0.649$\pm$0.083	10.42
(5.0, 20.0)	7.89e-07$\pm$5e-08	0.047$\pm$0.003	0.76